

Serverless Contact Form API using AWS Lambda, API Gateway & DynamoDB

Lab Title:

Serverless Contact Form API with Lambda, API Gateway & DynamoDB

Objective:

The objective of this lab is to build a **serverless REST API** that handles contact form submissions without managing any servers.

The project demonstrates how to integrate **API Gateway**, **AWS Lambda**, and **DynamoDB** to create a scalable, fully managed backend.

Services Used

- **Amazon API Gateway** – Handles incoming HTTP requests
 - **AWS Lambda** – Executes backend logic
 - **Amazon DynamoDB** – Stores form submission data
 - **IAM** – Manages permissions and access control
-

Skills Covered

- Writing Lambda functions in **Python**
 - Connecting **API Gateway** to **Lambda**
 - Performing **CRUD** operations with DynamoDB
 - Handling **JSON event payloads** and responses
 - Applying **IAM policies** for fine-grained permissions
-

Architecture Overview

Flow:

[Client / HTML Form]



[API Gateway]



[Lambda Function]

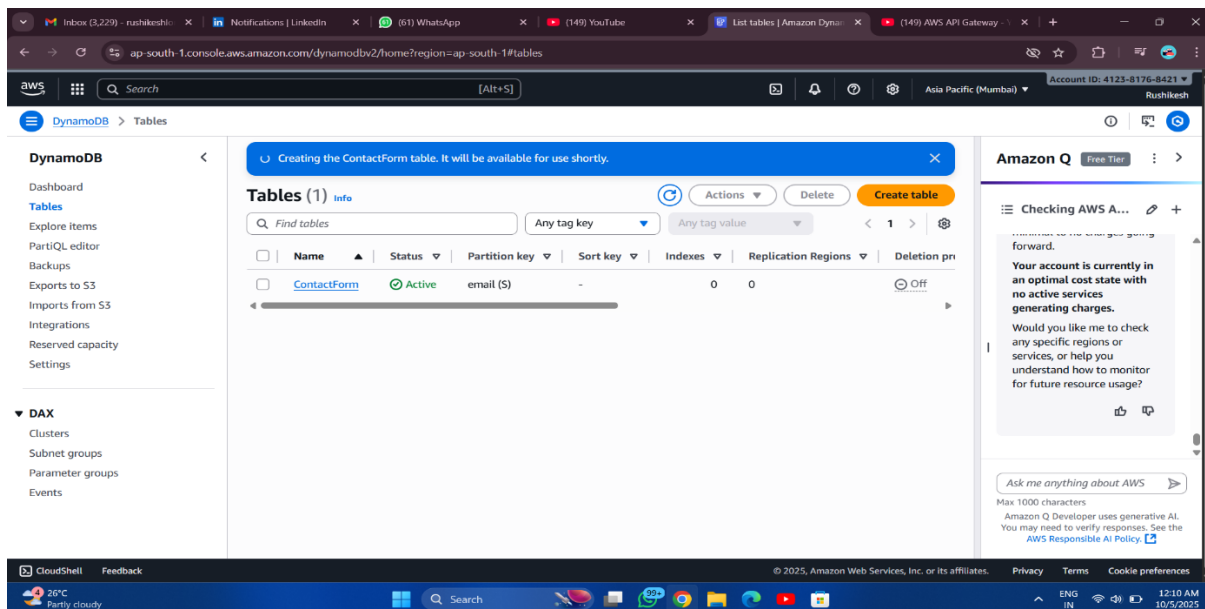


[DynamoDB Table]

Step-by-Step Implementation

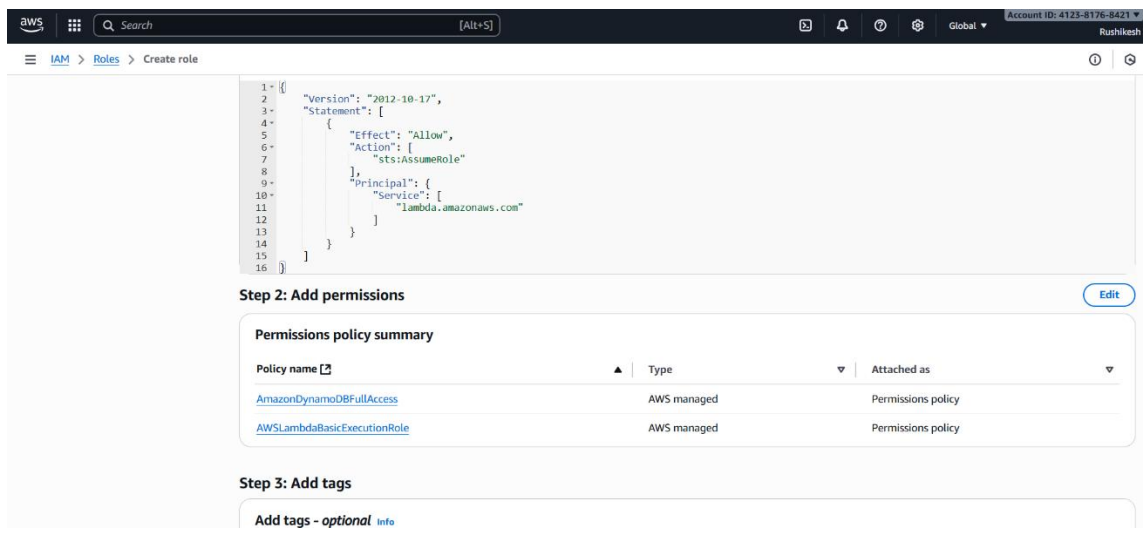
Step 1: Create DynamoDB Table

1. Go to **DynamoDB Console** → Click **Create Table**
2. **Table name:** ContactForm
3. **Partition key:** email (String)
4. Keep default settings and click **Create Table**



Step 2: Create IAM Role for Lambda

1. Open **IAM Console** → **Roles** → **Create Role**
2. Choose **AWS Service** → **Lambda**
3. Attach Policies:
 - **AWSLambdaBasicExecutionRole**
 - **AmazonDynamoDBFullAccess** (for this lab)
4. Give it a name: **LambdaDynamoDBRole**



Step 3: Create Lambda Function

1. Navigate to **AWS Lambda Console** → Click **Create Function**
2. **Name:** SubmitContactForm
3. **Runtime:** Python 3.12
4. **Execution Role:** Use the IAM role created above
5. **Code:**

```
import json
```

```
import boto3
```

```
from datetime import datetime
```

Rushikesh Londhe

```
dynamodb = boto3.resource('dynamodb')
```

```
table = dynamodb.Table('ContactForm')
```

```
def lambda_handler(event, context):
```

```
    body = json.loads(event['body'])
```

```
    name = body.get('name')
```

```
    email = body.get('email')
```

```
    message = body.get('message')
```

```
    if not all([name, email, message]):
```

```
        return {
```

```
            'statusCode': 400,
```

```
            'body': json.dumps({'error': 'All fields are required'})
```

```
        }
```

```
    # Store in DynamoDB
```

```
    table.put_item(Item={
```

```
        'email': email,
```

```
        'name': name,
```

```
        'message': message,
```

```
        'submitted_at': datetime.utcnow().isoformat()
```

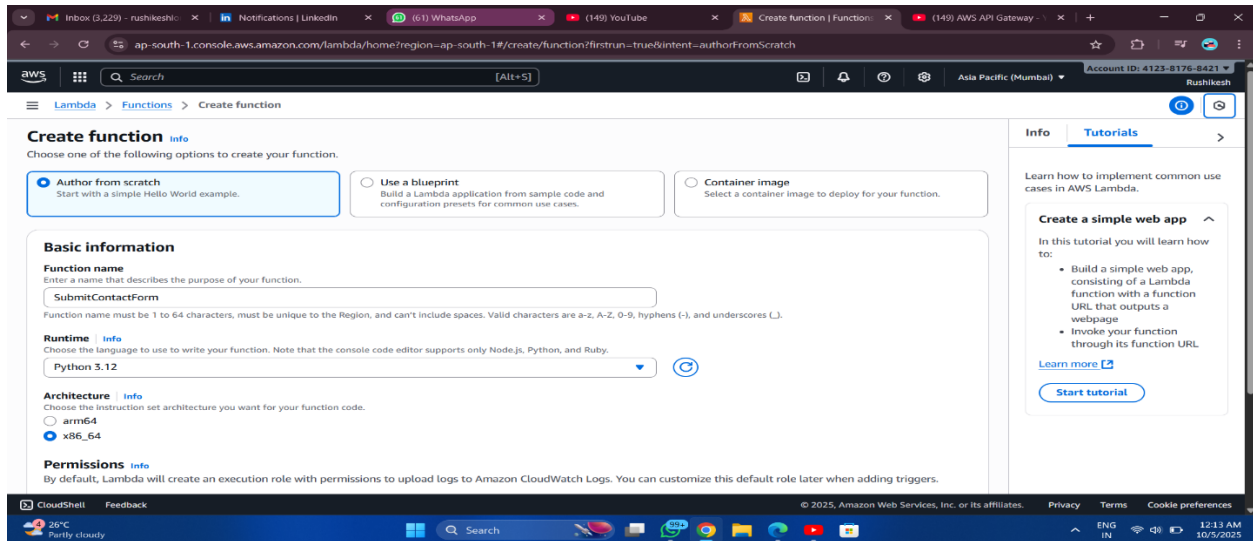
```
    })
```

```
    return {
```

```
        'statusCode': 200,
```

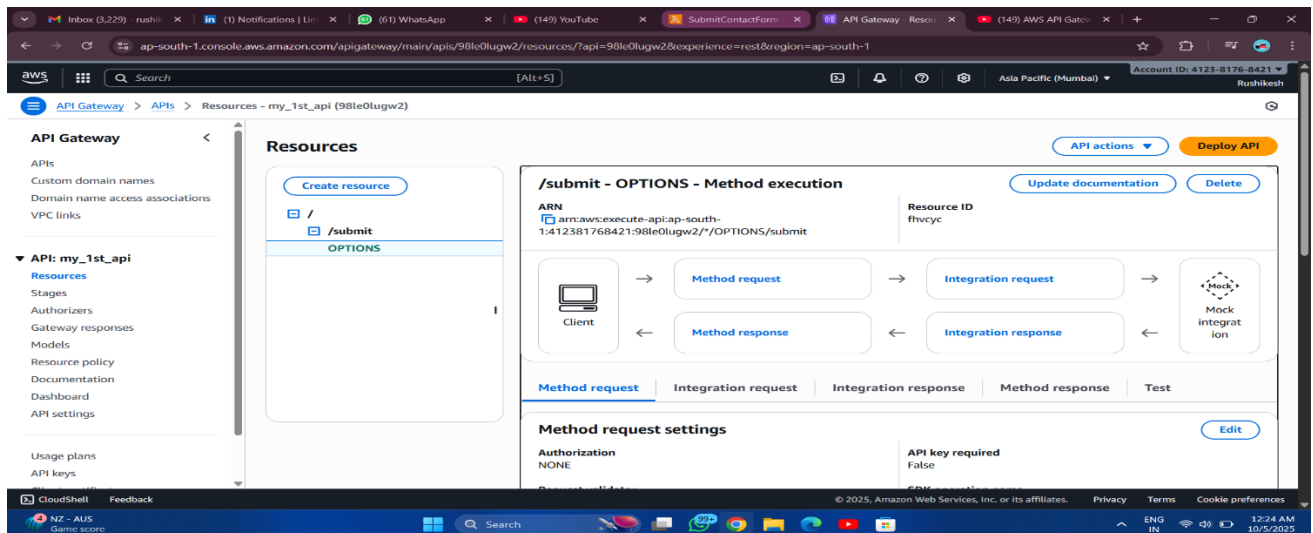
```
        'body': json.dumps({'message': 'Form submitted successfully'})
```

```
}
```



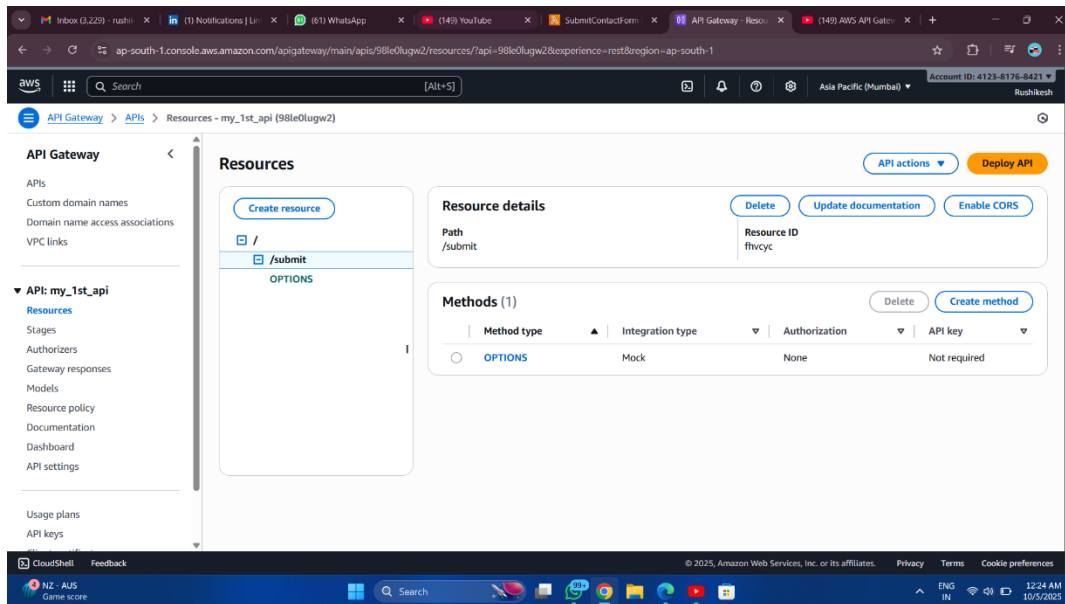
Step 4: Create API Gateway Endpoint

1. Open **API Gateway Console** → **Create API** → **REST API**
2. Create a **Resource** named `/submit`
3. Add a **POST Method**
 - **Integration type:** Lambda Function
 - **Select:** SubmitContactForm
4. **Enable CORS** (for browser testing)



Step 5: Deploy the API

1. Go to **Actions** → **Deploy API**
2. **Create New Stage:** Name it prod
3. Copy the **Invoke URL**, for example:
4. `https://<api-id>.execute-api.<region>.amazonaws.com/prod/submit`



Step 6: Test the API

✓ Using curl or Postman

Example command:

```
curl -X POST \
  https://9g0ean26k3.execute-api.us-east-1.amazonaws.com/prod/submit \
  -H "Content-Type: application/json" \
  -d '{"name":"Anhishek","email":"vicky.123531@gmail.com","message":"bye ji!"}'
```

Expected Response:

```
{"message": "Form submitted successfully"}
```

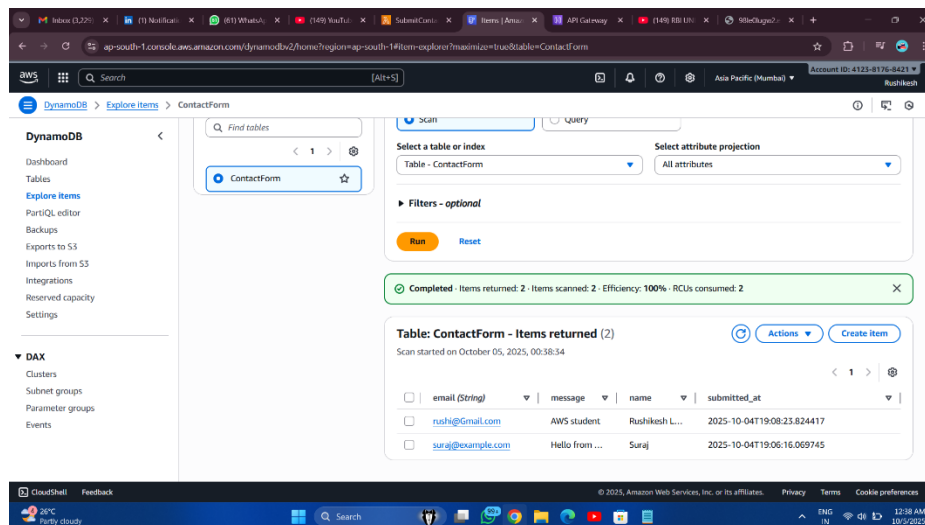
```
Pretty-print ☐

{"message": "Form submitted successfully"}
```

Step 7: Verify Data in DynamoDB

1. Go to **DynamoDB** → **ContactForm** → **Explore Table Items**
2. You should see records like:

```
{  
  "email": "Rushikesh Londhe",  
  "name": "Rushi@gmail.com",  
  "message": "aws student",  
  "submitted_at": "2025-10-05T18:45:00Z"  
}
```



Test with HTML Form

Rushikesh Londhe

Create a local HTML file (contact.html) with the following code:

```
<form action="https://9g0ean26k3.execute-api.us-east-1.amazonaws.com/prod/submit"
method="POST">
```

```
<input name="name" value="Suraj" />
```

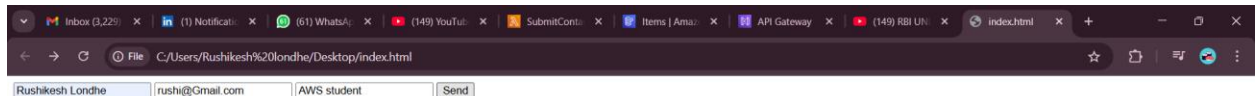
```
<input name="email" value="suraj@example.com" />
```

```
<input name="message" value="Hello from HTML!" />
```

```
<button type="submit">Send</button>
```

```
</form>
```

- Open this file in a browser and click **Send**
- Verify in DynamoDB if the entry is created successfully



•



Cleanup

To avoid unwanted charges:

- Delete **API Gateway**
- Delete **Lambda Function**

- Delete **DynamoDB Table**
 - Delete **IAM Role (if created only for this lab)**
-

Summary

You have successfully built a **Serverless REST API** that:

- Accepts HTTP POST requests through **API Gateway**
- Executes logic using **AWS Lambda**
- Stores form data securely in **DynamoDB**

This project demonstrates a complete **serverless architecture** suitable for modern cloud-based web applications.